Timeline: Adding the time dimension to spreadsheets

ANONYMOUS AUTHOR(S)

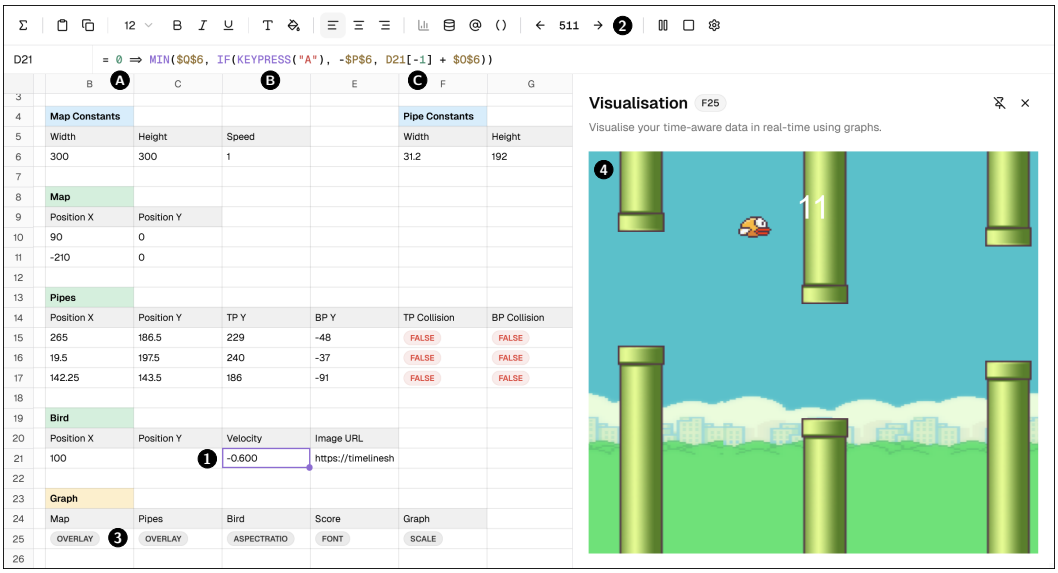

Fig. 1. A flappy bird game clone in Timeline. ❶ The selected formula computes the Y velocity in the current step. ❷ The system uses discrete time. ❸ Visualizations are built using a composable grammar of graphics and ❹ can be displayed inline or in a separate panel. The formula for the velocity Ⓐ specifies an initial value, Ⓑ reacts to user-interface events, and Ⓒ accesses a previous value to compute the current one.

Spreadsheets make it easy to express computations over two-dimensional data, but two dimensions are not enough to express rich computations with time such as simulations, complex systems models, analyses of historical data, or interactive systems. We present Timeline, a system that adds discrete time to spreadsheets.

The remarkable insight from our work is that a large number of advanced programming language concepts can be directly applied in the context of spreadsheets with time. Timeline draws from *dataflow languages* to express computations over time, *coeffect systems* to store only the necessary previous values, *functional reactive programming* to support interactivity, *grammars of graphics* to support composable visualizations, and *typed holes* for inserting cell references in the formula editor.

In this paper, we provide an overview of the Timeline design and discuss how it adapts the aforementioned programming language innovations for the context of spreadsheets. We then formalize the evaluation of spreadsheets with discrete time through a *core calculus*, describe a coeffect-based inference for computing the required number of past values and prove that the optimization is sound. More broadly, this paper shows that established programming language ideas can often be productively used outside of their original domain.

CCS Concepts: • **Software and its engineering → Very high level languages**.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1557-735X/2026/8-ART111
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Spreadsheets are sometimes referred to as "the most widely used programming systems in the world" [2]. Their uses range from personal productivity tools and decision making support systems to scientific analytical instruments and financial reporting systems [18]. However, spreadsheets are first-order, limited to tabular data structures, and lack abstraction mechanisms.

Many extensions to the core spreadsheet model have been proposed, generalising the tabular structure to non-tabular forms [6, 10], adding support for objects [28], or incorporating support for streaming data to use spreadsheets as the back-end for web development [11, 12]. Abstraction mechanism developed for spreadsheets include sheet-defined functions [24, 34] and gridlets [23], while programming by example [19] simplifies data wrangling within spreadsheets.

As the above extensions show, extending spreadsheets is a difficult balancing act. How can we make spreadsheets more expressive, while preserving their liveness, directness and usability [21]? The aim of the Timeline project is to extend spreadsheets with a built-in notion of discrete time, making it possible to use spreadsheets in a range of domains that are difficult to tackle with only two dimensions [25] such as physics simulations, agent-based modelling [1], financial data analysis, or simple interactive applications. As Figure 1 shows, Timeline adds an implicit discrete time to the spreadsheet programming model and extends the formula language to allow references to previous values and user interface events, but otherwise preserves the standard spreadsheet model.

Adapting programming langauge research. Although Timeline is interesting as an exploration of the design, expressive power and usability of spreadsheets with discrete time, this paper considers it primarily from the technical perspective of programming language design and implementation. The remarkable result is that a large number of concepts and techniques developed in the context of programming language research are directly applicable to spreadsheets.

The key addition to Timeline is a built-in notion of discrete time. Formulas in cells can refer to other cells in the current time step (e.g. `G19`), but also values computed in earlier steps (e.g. `F19[-1]`). The language design and runtime implementation of Timeline draws inspiration from research on *synchronous dataflow languages* [7, 20]. To ensure that simulations and interactive applications execute using bounded memory whenever possible, we augment the runtime with a program analysis based on *coeffects* [32, 33]. The support for discrete times also makes it possible to use the spreadsheet paradigm for building interactive applications. Here, we draw on the *functional reactive programming* paradigm [16, 36]. Although not strictly related to time, Timeline also adopts a *composable data visualization* model [26, 31, 37]. Finally, Timeline also uses a notion of *typed holes* [30] to determine when the user may want to insert a cell reference during formula editing.

Contributions. This paper presents the design and implementation of a spreadsheet system with discrete time and captures its essence through a core calculus.

- We provide an overview of the Timeline system design through a simple physics simulation example (§2) and examine the expressive power of the programming model through a number of case studies covering agent-based modelling, simulations and finance (§5).
- We discuss how how the design and implementation of Timeline adapts a range of advanced programming language concepts for the context of spreadsheets (§3), including dataflow languages, coeffects, functional reactive programming, visualization grammars and typed holes.
- We present a core calculus of the Timeline execution model (§4), which captures the way previous values are stored during the spreadsheet evaluation. We define a mechanism for checking references to earlier time steps (§4.4), formalize a space optimization based on coeffects (§4.5) and prove their soundness (§4.6).

2 Overview

Although the focus of this paper is on the programming language aspects of Timeline, we first briefly review the design motivation for adding time to spreadsheets. We then introduce Timeline through an example that leverages the time dimension and illustrates the capabilities of the system.

Design Motivation. To motivate the design of Timeline, we conducted formative interviews with spreadsheet experts, including some with major experience [15], and we analysed models created using modelling languages with discrete time [35]. The analysis shows that tasks involving discrete time often require three dimensions. In agent-based modeling [1], a simulation consists of multiple agents with multiple attributes that evolve over time. Physics simulations that require numerical solution [35] also require tracking multiple attributes of multiple entities over time. Financial analyses of historical data have the same structure.

Although conceptually simple, such modelling problems are difficult to represent in the traditional spreadsheet paradigm. Using one dimension for time forces users to seek workarounds for encoding the remaining two dimensions, for example by splitting attributes of individual agents across separate tables. In practice, analysts are reluctant to use one of the two spreadsheet dimensions for time. This has been confirmed by a participant in the formative interview, who noted that *"once you set the rows to time steps, you have lost a dimension of your spreadsheet"* and *"if you use one [dimension] for time, everything else becomes miserable."*

Motivating Example. To demonstrate Timeline, we consider a simple numerical physics simulation of cannonball firing. In each step, the simulation computes the cannonball position using the position in the previous step and the current velocity. The initial velocity is computed from the initial angle and firing speed and then updated using gravity and air drag, which itself depends on velocity in the previous step. The simulation runs until the cannonballs hit the ground.

There are three dimensions in the example: we have multiple cannonballs, with multiple attributes, updated over multiple time steps. Implementing the simulation in a conventional spreadsheet is possible, but challenging. One option is to have multiple tables, one for each attribute (X and Y positions, X and Y velocities, X and Y air drags) with columns for individual cannonballs and rows for time steps. This results in a complex structure with computation spread across 8 tables, but it allows adding cannonballs and time steps using drag right and drag down operations (the user selects the existing table and drags the selection to copy formulas to further empty cells).

Implementing Cannonball Simulation. In Timeline, we can implement the cannonball simulation using a single table. We use this example to introduce the key features of the system.

The calculation of projectile motion is done in a single table that records attributes of individual cannonballs over time (Figure 2, ❶). The table includes the current position (`B3`, `C3`), velocity (`F3`, `G3`) and air drag (`H3`, `I3`) as well as the initial angle and speed (`D3`, `E3`). We discuss the calculation for the first cannonball (row index 3). Other cannonballs are added by selecting the table row and using the drag down operation to copy its formulas, automatically updating the row references.

To specify the position of a cannonball, we need to give the initial position and then specify how to compute the position in the current time step in terms of the value in the previous time step. This illustrates two extensions that Timeline makes to the core spreadsheet model. First, the => operator specifies an initial value for a cell. Second, the notation `C3[-1]` lets us access the value of the cell `C3` in a previous time step. For cannonball, the initial X and Y positions are 0, the position in later steps is computed by adding the current X and Y velocities to the previous position and the computation stops if the cannonball hits the ground:

```
/* Position X */ B3 = 0 => IF(C3[-1] < 0, B3[-1], B3[-1] + F3)
/* Position Y */ C3 = 0 => IF(C3[-1] < 0, C3[-1], C3[-1] + G3)
```

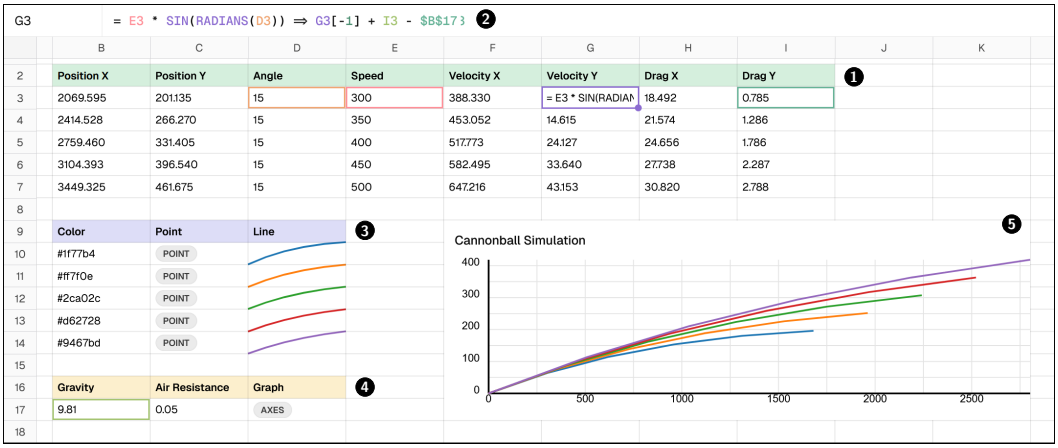

Fig. 2. Numerical cannonball firing simulation in Timeline. ❶ Single table records attributes of multiple cannonballs over time. ❷ The attributes are computed based on current and previous values of other attributes. ❸ A separate table plots past positions as lines. ❹ A table with configuration and composed final chart. ❺ The composed chart is then embedded in the sheet.

The calculation of the cannonball velocity again uses =>. This time, the initial value is a formula using the firing speed and angle (E3, D3). The velocity is then computed from the value in the previous time step and air drag. For the Y velocity, we also subtract gravity, which is a fixed cell reference (B17) that is not updated during the drag down operation. Air drag is obtained by multiplying the previous X and Y velocities by the air resistance (C17):

```
/* Velocity X */ F3 = E3 * COS(RADIANS(D3)) => F3[-1] + H3
/* Velocity Y */ G3 = E3 * SIN(RADIANS(D3)) => G3[-1] + I3 - $B$17
/* Air Drag X */ H3 = F3[-1] * $C$17
/* Air Drag Y */ I3 = G3[-1] * $C$17
```

Note that the calculation of air drag does not require specifying an initial value, because there are no references to previous values of H3 and I3. The only references to the cells are in the velocity computation, but those refer to the current value. The air drag computation accesses velocity at the previous time step, which always exists because velocity defines an initial value using =>. As in standard spreadsheet systems, references between current values of cells need to be acyclic. References to previous time steps do not introduce a cycle.

An interesting problem is how many past values need to be kept by the system. In the computation described so far, we access one previous value of cells B3, C3 and F3, G3 using the [-1] notation. It is thus sufficient to keep one past value for these four cells. For the chart (Figure 2, ❺) we need to store all previous values of the cells that define the line. We discuss this aspect in detail in §3.2.

Composable Data Visualization. Data visualizations in standard spreadsheet systems exist separately of the core spreadsheet model. They are created through graphical user interface where the user specifies data sources and other chart attributes. Timeline takes inspiration from research on spreadsheet-based and functional composable visualizations [26, 31] and allows constructing of visualizations directly in the spreadsheet.

The cannonball firing chart (Figure 2, ❺) is obtained by constructing line charts that represent the movement of individual cannonballs over time and composing the individual lines (❸ and ❹). This is done in a separate table that contains one chart for each cannonball. Table row at index 10 (C10, D10) constructs a line chart for cannonball at row index 3 (B3, C3):

```
/* Point */ C10 = POINT(B3, C3)
/* Line */  D10 = GRAPH(STROKECOLOR(B10, LINE(C10[:-STEP()])))
/* Graph */ D17 = AXES("left bottom", OVERLAY(D10:D14))
```

The example uses a function POINT that constructs a value representing a two-dimensional point and a number of functions for constructing and transforming charts including LINE, OVERLAY and AXES. The latter are inspired by composable data visualizations [31]. LINE takes a range of points and constructs a primitive chart shape representing a line; OVERLAY takes a range of chart shapes and constructs a single chart shape consisting of the individual shapes; AXES attaches axes with labels to a given chart shape. The addition requires adding points and chart shapes as two additional primitive values to the core spreadsheet model. In systems that support richer data structures [10, 14, 29, 39], a point could be a user-defined object or a record.

To construct the chart, the cell C10 constructs a point consisting of the current X and Y positions of the cannonball. We then want to construct a line from all previous locations of the cannonball. This is done by specifying a time range [:-STEP()] in the cell reference. The syntax is a shorthand for [-1:-STEP()] and it selects all previous steps (STEP is a function that returns the number of the current step). The array of past values is then passed to LINE, which constructs a primitive chart shape. The GRAPH function used in cell D10 instructs Timeline to display the chart inline in the cell as a sparkline [17] without modifying the shape value. Finally, the cell D17 then overlays the individual lines and adds axes. The underlying charting library automatically aligns the chart elements and computes the range.

Extending the Spreadsheet Paradigm. The preceding example illustrated most of the extensions that Timeline makes to the core model of spreadsheets. In fact, the only significant additional feature needed by the opening Flappy Bird clone (Figure 1) is the KEYPRESS("A") function, which detects whether a key has been pressed. The game movements and collision detection are ordinary formulas and the game scene is constructed using the composable data visualization library, leveraging the IMAGE function to construct a primitive chart shape from an image URL.

The motivating example shows how the additions made in Timeline preserve the simplicity and directness of the core spreadsheet model. We add only a limited number of extensions to the formula language (the => operator and references across time). Moreover, when creating a spreadsheet in Timeline, users still start with concrete values and then extend their formulas using the drag down operation to apply a formula across a range of inputs, preserving the *from concrete cases* mode of *abstraction construction* [22].

3 Language features

first say what we add => and 3D range selectors

3.1 Synchronous languages

3.2 Coeffects

3.3 Functional reactive programming

3.4 Composable data vis

3.5 Typed holes

Column name	N	$=$	A | B | C | ...	**Value**	V	$=$	$c \mid \{V_{i,j}\}$
Row index	m	$\in$	$\mathbb{N}_1$	**Formula**	F	$=$	$V \mid R \mid f(F_1, \ldots, F_n)$
Address	a	$=$	Nm	**Sheet**	σ	$\in$	$Address \rightarrow Formula$
Range	R	$=$	$a_1 : a_2$	**Grid**	γ	$\in$	$Address \rightarrow Value$

Fig. 3. Syntax of the core Timeline calculus.

Formula evaluation: $\sigma \vdash F \Downarrow V$

$$\mathsf{size}(N_1 m_1 : N_2 m_2) = (N_2 - N_1 + 1, m_2 - m_1 + 1)$$
$$N_1 m_1 + (i, j) = (N_1 + i - 1)(m_1 + j - 1)$$

$$\frac{}{\sigma \vdash V \Downarrow V} \qquad \frac{\sigma \vdash F_i \Downarrow V_i \quad [\![f]\!](V_1, \ldots, V_n) = V}{\sigma \vdash f(F_1, \ldots, F_n) \Downarrow V} \qquad \frac{\sigma \vdash a \,!\, V}{\sigma \vdash a : a \Downarrow V}$$

$$\frac{a_1 \neq a_2 \quad \mathsf{size}(a_1 : a_2) = (n, m) \quad \forall i \in 1..n, j \in 1..m \,.\, \sigma \vdash (a_1 + (i, j)) \,!\, V_{i,j}}{\sigma \vdash a_1 : a_2 \Downarrow \{V_{i,j}\}}$$

Address dereferencing: $\sigma \vdash a \,!\, V$

$$\frac{\sigma(a) = F \quad \sigma \vdash F \Downarrow V}{\sigma \vdash a \,!\, V}$$

Sheet evaluation: $\sigma \Downarrow \gamma$

$$\frac{\forall a \in \mathsf{dom}(\sigma) \,.\, \gamma(a) = V \wedge \sigma \vdash \sigma(a) \Downarrow V}{\sigma \Downarrow \gamma}$$

Fig. 4. Reduction rules that define formula evaluation of the core Timeline calculus.

4 Calculus

In this section, we present a core calculus of Timeline. We take the *core calculus for spreadsheets* [38] as our starting point (§4.1, §4.2) and extend it with the notion of time (§4.3). We then define a system for checking references to earlier time steps (§4.4), formalize our coeffect-based optimization (§4.5) and show their soundness (§4.6).

4.1 Syntax

Figure 3 defines the syntax of the calculus. Addresses Nm consist of a column name (such as A, B, or AB) and a row index (starting from 1) and ranges are pairs of addresses. Values can be constants c or a two-dimensional arrays $\{V_{i,j}\}$ with indices $i \in 1..n$, $j \in 1..m$.

In contrast to prior work [9, 38], we omit empty values, error values and implicit error propagation. We treat errors as constants and assume that built-in functions return error when called with the error value as argument. Although we do not support spilling [38], we include arrays of values and later extend those to support ranges over time. Note also that a singleton array $\{V\}$ is distinct from a primitive value V. Formulas can be primitive values V, range references R or built-in function calls. For simplicity, cells can only be referenced through ranges so a reference `B5` are modelled as a singleton range reference `B5:B5`.

Time	t	$\in$	$\mathbb{N}_1$	**Expression**	E	$=$	$V \mid R \mid f(E_1, \ldots, E_n)$
Interval	T	$=$	$E_1 : E_2$	**Formula**	F	$=$	$E \mid E \Rightarrow F$
Range	R	$=$	$a_1 : a_2 \mid a_1 : a_2[T]$	**Timeline**	ι	$\in$	$\{1 \ldots t\} \rightarrow \mathit{Value}$
Value	V	$=$	$\ldots \mid t \mid \{V_{t,i,j}\}$	**History**	ω	$\in$	$\mathit{Address} \rightarrow \mathit{Timeline}$

Fig. 5. Syntax of the Timeline calculus with time.

Following the terminology of Williams et al. [38], we use the term *sheet* to refer to the source code and *grid* for the result of evaluating a sheet. A sheet σ is a partial function from addresses to formulas, while a grid γ is a partial function from addresses to values.

4.2 Semantics

The evaluation rules are defined in Figure 4. We follow the style of Williams et al. [38] and define evaluation with respect to a an unevaluated sheet σ. This means that dereferencing an address, defined as an auxiliary relation $\sigma \vdash a\,!\,V$ obtains the formula of the target cell and evaluates it recursively. Consequently, evaluation of cyclic references is not defined. An alternative approach [9] defines evaluation with respect to (other) evaluated values as $\gamma \vdash F \Downarrow V$ and imposes consistency requirements on the recalculation process. This rules out cyclic references for which there is no consistent value assignment, such as A1 = A1 + 1, but permits self-references such as A1 = A1.

Formula evaluation $\sigma \vdash F \Downarrow V$ is then a relation that evaluates a formula F to a value V given the sheet definition σ. A value V evaluates to itself. A function call $f(F_1, \ldots, F_n)$ evaluates its arguments and then obtains the result by applying $[\![f]\!]$, which defines the semantics of the built-in function, to the arguments. Evaluating a singleton range $a : a$ obtains a single value at the given address using dereferencing. Evaluating a range $a_1 : a_n$ iterates over all addresses in the range. The helper size returns the size of the range and $a + (i, j)$ returns an address moved by 1-based offsets i and j.

Sheet evaluation $\sigma \Downarrow \gamma$ evaluates the value at each address $a \in \mathrm{dom}(\sigma)$ defined in the sheet σ. We do not specify evaluation order, but the evaluation is still deterministic (§4.6). Our Timeline implementation builds a dependency graph between cells and evaluates them in a topological order. As discussed earlier, sheet evaluation is not defined for sheets containing a cyclic reference.

4.3 Adding the time dimension

To model Timeline, we extend the core calculus with a notion of discrete time. The extended syntax is shown in Figure 5. Time t is modelled using positive integers and, following the Timeline implementation, we refer to history using negative values $-t$. We treat time as a kind of value V. A time interval T is defined by a pair of expressions that evalue to time values. A range R can now refer to a three-dimensional range over a sheet area and a time interval. A reference to a single past value such as `F19[-1]` is modelled as a singleton range `F19:F19[-1:-1]`. Evaluating a reference to a three-dimensional range produces a three-dimensional array of values, represented as a new kind of value written as $\{V_{t,i,j}\}$.

Note that intervals can only range over historical values and so we cannot write, for example, `SUM(F19[0:-1])` to add the current and the previous value of `F19`. This simplifies the model as previous and current values are stored differently. Corresponding computations can be expressed explicitly using, for example, `F19+F19[-1]`.

To model the *followed by* operator $\Rightarrow$, we separate formulas F and expressions E. A formula can be an expression, or a formula prefixed with an initial value (i.e., a formula has zero or more prefixed values). In addition to the previously defined grid γ, we maintain the history of previously evaluated values. A history ω is a partial function mapping addresses to timelines. A timeline stores a continuous range of t previous values of a cell, indexed by a time interval from 1 to t.

$\boxed{\text{Formula evaluation: } \sigma, \omega \vdash F \Downarrow V}$ $\boxed{\text{History access: } \omega \vdash a[-t] \,\text{¡}\, V}$

$$\frac{\sigma, \omega \vdash F \Downarrow V}{\sigma, \omega \vdash E \Rightarrow F \Downarrow V} \qquad \frac{\sigma, \omega \vdash E \Downarrow -t \qquad \omega \vdash a[-t] \,\text{¡}\, V}{\sigma, \omega \vdash a{:}a[E{:}E] \Downarrow V} \qquad \frac{\omega(a)(t) = F}{\omega \vdash a[-t] \,\text{¡}\, V}$$

$$\frac{\begin{array}{c}\sigma, \omega \vdash E_1 \Downarrow -t_1 \qquad \sigma, \omega \vdash E_2 \Downarrow -t_2 \\ a_1 \neq a_2 \vee E_1 \neq E_2 \qquad \mathsf{size}(a_1{:}a_2) = (n, m) \qquad l = t_2 - t_1 + 1 \\ \forall i \in 1..n, j \in 1..m, t \in 1..l \,.\, \omega \vdash a_1[-t_1] + (t, i, j) \,\text{¡}\, V_{t,i,j}\end{array}}{\sigma, \omega \vdash a_1{:}a_2[-t_1{:}-t_2] \Downarrow \{V_{t,i,j}\}}$$

$$N_1 m_1[-t_1] + (t, i, j) = (N_1 + i - 1)(m_1 + j - 1)[-(t_1 + t - 1)]$$

Fig. 6. Reduction rules that define formula evaluation of the Timeline calculus with time.

$\boxed{\text{Sheet evaluation: } \sigma, \omega \Downarrow \gamma}$

$$\frac{\forall a \in \mathsf{dom}(\sigma) \,.\, \gamma(a) = V \wedge \sigma, \omega \vdash \sigma(a) \Downarrow V}{\sigma, \omega \Downarrow \gamma}$$

$\boxed{\text{Iterated evaluation: } \sigma \Downarrow_n \gamma}$

$$\begin{array}{ll}
\sigma \Downarrow_n \gamma_n \iff \sigma, \omega_{n-1} \Downarrow \gamma_n & \text{where } \forall k \in 0 \ldots n{-}1 \\
\omega_0(a) = \{1 \mapsto V_1, \ldots, n \mapsto V_n\} & \text{where } \sigma(a) = E_n \Rightarrow \ldots \Rightarrow E_1 \Rightarrow E \text{ and } \emptyset, \emptyset \vdash E_i \Downarrow V_i \\
\omega_{k+1}(a)(1) = \gamma_{k+1}(a) & \text{where } \sigma, \omega_k \Downarrow \gamma_{k+1} \\
\omega_{k+1}(a)(1 + t) = \omega_k(t) &
\end{array}$$

Fig. 7. Sheet evaluation and iterated sheet evaluation for Timeline.

Formula evaluation. Figure 6 defines the extended semantics. Formula evaluation is now defined with respect to the sheet σ but also previously evaluated values represented using a history ω. The additional context can be easily added to the rules in Figure 6, but we define new rules for three-dimensional ranges and formulas prefixed with initial values. The value of $E \Rightarrow F$ is the value of F. The initial value will be used later, when initializing the history ω.

To access past values, we define an auxiliary relation $\omega \vdash a[-t] \,\text{¡}\, V$ akin to address dereferencing. However, the relation is defined with respect to the previously evaluated values and it accesses the past value from the timeline, rather than evaluating it.

In general, time ranges are specified by expressions that need to be evaluated before accessing past values. The evaluation again distinguishes the case when a range (syntactically) refers to a single value. If so, the value is obtained from the history and returned. For other ranges, we iterate over all the values and build a three-dimensional array $\{V_{t,i,j}\}$. We extend our earlier auxiliary definition of + to also support computing an offset for an address at a specified time.

Note also that some built-in functions are time-dependent. In particular, `STEP()` returns the number of the current time step. To model this, we would need to supply additional information to the semantic function $[\![-]\!]$. We omit the details for simplicity.

$$\frac{}{\Omega \vdash V \text{ ok}} \qquad \frac{}{\Omega \vdash a_1 : a_2 \text{ ok}} \qquad \frac{\Omega \vdash E_i \text{ ok}}{\Omega \vdash f(E_1, \ldots, E_n) \text{ ok}}$$

$$\frac{\Omega(a) \geq t}{\Omega \vdash a[-t] \text{ ok}} \qquad \frac{\forall a_i \in \text{expand}(a_1 : a_2).\, \Omega \vdash a_i[-t] \text{ ok} \quad t = \max(t_1, t_2)}{\Omega \vdash a_1 : a_2[-t_1 : -t_2] \text{ ok}}$$

$$\frac{\Omega(a) = \infty}{\Omega \vdash a[E_1 : E_2] \text{ ok}} \qquad \frac{\forall a_i \in \text{expand}(a_1 : a_2).\, \Omega \vdash a_i[E_1 : E_2] \text{ ok} \quad (E_1 \neq t_1) \vee (E_2 \neq t_2)}{\Omega \vdash a_1 : a_2[E_1 : E_2] \text{ ok}}$$

$$\text{expand}(a_1 : a_2) = \{a_1 + (i, j), \forall i \in 1..n, j \in 1..m\} \text{ where } (n, m) = \text{size}(a_1 : a_2)$$

Fig. 8. Checking

Sheet evaluation. Ordinary spreadsheets are evaluated to a grid of values using the $\sigma \Downarrow \gamma$ relation (Figure 4). In Timeline, the evaluation proceeds iteratively in discrete time steps for as long as the user desires (Figure 7). In each step, the sheet is evaluated using the sheet evaluation $\sigma, \omega \Downarrow \gamma$, relation which computes the current values of the formulas, with respect to a given history. This relation remains unchanged from the ordinary version, except that it now provides the current history ω to the formula evaluation.

The iterated evaluation $\sigma \Downarrow_n \gamma$ models the first n step of the evaluation of a Timeline spreadsheet. It first constructs the initial history ω_0 which is obtained by evaluating the default values (expressions before the => operator) for each cell. The number of default values corresponds to the number of `E =>` prefixes in the formula of the cell. Note that the default values are evaluated with an empty sheet and an empty history as the context. That is, an expression that defines a default value cannot itself reference other cells or their historical values (the semantics could be extended to allow referencing of other cells with greater or equal number of default values).

After constructing the initial history ω_0, the iterated evaluation constructs a sequence of histories $\omega_1, \ldots, \omega_{n-1}$ such that the previous value of a cell in step $k + 1$ becomes the value in the evaluated grid in step k and all earlier past values are shifted by 1.

4.4 Checking timed accesses

The problem of type inference and type checking for spreadsheets has been adressed by others [3, 5] and their approaches could be directly applied to Timeline. A problem unique to Timeline is ensuring that sufficient number of past values is available.

see Figure 8

with this, we could drop (old) values from the history and also make sure there is enough default values

TODO: But we do not have infinity in ω_0 yet

4.5 Coeffect optimization

before

$$\frac{\Omega(a) \geq t_0 + t_1}{\Omega \mathbin{@} t_0 \vdash a[-t_1]\ \mathsf{ok}} \qquad \frac{\Omega(a) = \infty}{\Omega \mathbin{@} t_0 \vdash a[E_1 : E_2]\ \mathsf{ok}}$$

after

$$\frac{a \notin \mathsf{dom}(\Omega) \quad \Omega \mathbin{@} t_0 + t_1 \vdash_\sigma \sigma(a)\ \mathsf{ok}}{\Omega \mathbin{@} t_0 \vdash_\sigma a[-t_1]\ \mathsf{ok}} \qquad \frac{a \notin \mathsf{dom}(\Omega) \quad \Omega \mathbin{@} \infty \vdash_\sigma \sigma(a)\ \mathsf{ok}}{\Omega \mathbin{@} t_0 \vdash_\sigma a[E_1 : E_2]\ \mathsf{ok}}$$

??

funny history access

$$\frac{\sigma, \omega(a)(t) = F}{\omega \vdash a[-t] \mathbin{¡} V}$$

x

$$\mathsf{shift}_t(\omega)(a)(t') = \omega(a)(t + t')$$

x

$$\frac{\omega(a)(t) = \bot \qquad \sigma(a) = F \qquad \sigma, \mathsf{shift}_t(\omega) \vdash F \Downarrow V}{\sigma, \omega \vdash a[-t] \mathbin{¡} V}$$

check that all items accessed at previous time step have enough past values specified

4.6 Properties and coeffect soundness

sheet evaluation is deterministic

5 Case studies

5.1 Formative interviews

6 Related work

static analysis [13] type inference [4]

sheet defined functions [24] [34] and their semantics [9] elastic [27]

gridlets [23] and their semantics [38] [8]

References

[1] Sameera Abar, Georgios K. Theodoropoulos, Pierre Lemarinier, and Gregory M.P. O'Hare. 2017. Agent Based Modelling and Simulation tools: A review of the state-of-art software. *Computer Science Review* 24 (2017), 13–33. doi:10.1016/j.cosrev.2017.03.001

[2] Robin Abraham, Margaret Burnett, and Martin Erwig. 2009. *Spreadsheet Programming*. John Wiley & Sons, Ltd, 2804–2810. doi:10.1002/9780470050118.ecse415

[3] Robin Abraham and Martin Erwig. 2006. Type inference for spreadsheets. In *Proceedings of the 8th ACM SIGPLAN International Conference on Principles and Practice of Declarative Programming* (Venice, Italy) *(PPDP '06)*. Association for Computing Machinery, New York, NY, USA, 73–84. doi:10.1145/1140335.1140346

[4] Robin Abraham and Martin Erwig. 2006. Type inference for spreadsheets. In *Proceedings of the 8th ACM SIGPLAN International Conference on Principles and Practice of Declarative Programming* (Venice, Italy) *(PPDP '06)*. Association for Computing Machinery, New York, NY, USA, 73–84. doi:10.1145/1140335.1140346

[5] Y. Ahmad, T. Antoniu, S. Goldwater, and S. Krishnamurthi. 2003. A type system for statically detecting spreadsheet errors. In *18th IEEE International Conference on Automated Software Engineering, 2003. Proceedings*. 174–183. doi:10.1109/ASE.2003.1240305

[6] Pavel Bažant and Michaela Maršálková. 2018. A Non-Tabular Spreadsheet with Broad Applicability. In *Companion Proceedings of the 2nd International Conference on the Art, Science, and Engineering of Programming (Programming '18)*. Association for Computing Machinery, New York, NY, USA, 161–165. doi:10.1145/3191697.3214343

[7] A. Benveniste, P. Caspi, S.A. Edwards, N. Halbwachs, P. Le Guernic, and R. de Simone. 2003. The synchronous languages 12 years later. *Proc. IEEE* 91, 1 (2003), 64–83. doi:10.1109/JPROC.2002.805826

[8] Alexander Asp Bock, Thomas Bøgholm, Peter Sestoft, Bent Thomsen, and Lone Leth Thomsen. 2022. On the cost semantics for spreadsheets with sheet-defined functions. *J. Comput. Lang.* 69 (2022), 101103. doi:10.1016/J.COLA.2022.101103

[9] Alexander Asp Bock, Thomas Bøgholm, Peter Sestoft, Bent Thomsen, and Lone Leth Thomsen. 2020. On the semantics for spreadsheets with sheet-defined functions. *Journal of Computer Languages* 57 (2020), 100960. doi:10.1016/j.cola.2020.100960

[10] MARGARET BURNETT, JOHN ATWOOD, REBECCA WALPOLE DJANG, JAMES REICHWEIN, HERKIMER GOTTFRIED, and SHERRY YANG. 2001. Forms/3: A first-order visual language to explore the boundaries of the spreadsheet paradigm. *Journal of Functional Programming* 11, 2 (2001), 155–206. doi:10.1017/S0956796800003828

[11] Kerry Shih-Ping Chang and Brad A. Myers. 2014. Creating Interactive Web Data Applications with Spreadsheets. In *Proceedings of the 27th Annual ACM Symposium on User Interface Software and Technology (UIST '14)*. Association for Computing Machinery, New York, NY, USA, 87–96. doi:10.1145/2642918.2647371

[12] Kerry Shih-Ping Chang and Brad A. Myers. 2015. A Spreadsheet Model for Handling Streaming Data. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems (CHI '15)*. Association for Computing Machinery, New York, NY, USA, 3399–3402. doi:10.1145/2702123.2702587

[13] Tie Cheng and Xavier Rival. 2015. Static Analysis of Spreadsheet Applications for Type-Unsafe Operations Detection. In *Programming Languages and Systems*, Jan Vitek (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 26–52.

[14] Chris Clack and Lee Braine. 1997. Object-oriented functional spreadsheets. In *Proceedings of the 10th Glasgow Workshop on Functional Programming (GlaFP'97)*.

[15] Department for Energy Security & Net Zero. 2020. *The MacKay Carbon Calculator*. https://www.gov.uk/guidance/carbon-calculator Last updated 21 August 2024. Accessed: 25 February 2026.

[16] Conal Elliott and Paul Hudak. 1997. Functional reactive animation. In *Proceedings of the Second ACM SIGPLAN International Conference on Functional Programming* (Amsterdam, The Netherlands) *(ICFP '97)*. Association for Computing Machinery, New York, NY, USA, 263–273. doi:10.1145/258948.258973

[17] Leo D. Frishberg. 2011. Interactive sparklines: a dynamic display of quantitative information. In *CHI '11 Extended Abstracts on Human Factors in Computing Systems* (Vancouver, BC, Canada) *(CHI EA '11)*. Association for Computing Machinery, New York, NY, USA, 589–604. doi:10.1145/1979742.1979655

[18] Thomas A Grossman and Vijay Mehrotra. 2023. A Use Case-Engineering Resources Taxonomy for Analytical Spreadsheet Models. In *21st EuSpRIG Annual Conference "The Spreadsheet Crisis: Regaining Control"*.

[19] Sumit Gulwani, William R. Harris, and Rishabh Singh. 2012. Spreadsheet Data Manipulation Using Examples. *Commun. ACM* 55, 8 (Aug. 2012), 97–105. doi:10.1145/2240236.2240260

[20] N. Halbwachs, P. Caspi, P. Raymond, and D. Pilaud. 1991. The synchronous data flow programming language LUSTRE. *Proc. IEEE* 79, 9 (1991), 1305–1320. doi:10.1109/5.97300

[21] Felienne Hermans, Bas Jansen, Sohon Roy, Efthimia Aivaloglou, Alaaeddin Swidan, and David Hoepelman. 2016. Spreadsheets are Code: An Overview of Software Engineering Approaches Applied to Spreadsheets. In *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, Vol. 5. 56–65. doi:10.1109/SANER.2016.86

[22] Joel Jakubovic, Jonathan Edwards, and Tomas Petricek. 2023. Technical Dimensions of Programming Systems. *The Art, Science, and Engineering of Programming* 7, 3 (2023), 13:1–13:59. doi:10.22152/programming-journal.org/2023/7/13

[23] Nima Joharizadeh, Advait Sarkar, Andrew D. Gordon, and Jack Williams. 2020. Gridlets: Reusing Spreadsheet Grids. In *Extended Abstracts of the 2020 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) *(CHI EA '20)*. Association for Computing Machinery, New York, NY, USA, 1–7. doi:10.1145/3334480.3382806

[24] Simon Peyton Jones, Alan Blackwell, and Margaret Burnett. 2003. A user-centred approach to functions in Excel. In *Proceedings of the Eighth ACM SIGPLAN International Conference on Functional Programming* (Uppsala, Sweden) *(ICFP '03)*. Association for Computing Machinery, New York, NY, USA, 165–176. doi:10.1145/944705.944721

[25] Steve Litt. 2017. Mitigating Spreadsheet Risk in Complex Multi-Dimensional Models in Excel. In *Proceedings of the EuSpRIG 2017 Conference "Spreadsheet Risk Management"*. European Spreadsheet Risks Interest Group (EuSpRIG), London, UK.

[26] Mariana Marasoiu, Detlef Nauck, and Alan F. Blackwell. 2019. Cuscus: An End User Programming Tool for Data Visualisation. In *End-User Development*, Alessio Malizia, Stefano Valtolina, Anders Morch, Alan Serrano, and Andrew Stratton (Eds.). Springer International Publishing, Cham, 115–131.

[27] Matt McCutchen, Judith Borghouts, Andrew D. Gordon, Simon Peyton Jones, and Advait Sarkar. 2020. Elastic sheet-defined functions: Generalising spreadsheet functions to variable-size input arrays. *Journal of Functional Programming* 30 (2020), e26. doi:10.1017/S0956796820000234

[28] Matt McCutchen, Shachar Itzhaky, and Daniel Jackson. 2016. Object Spreadsheets: A New Computational Model for End-User Development of Data-Centric Web Applications. In *Proceedings of the 2016 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! 2016)*. Association for Computing Machinery, New York, NY, USA, 112–127. doi:10.1145/2986012.2986018

[29] Matt McCutchen, Shachar Itzhaky, and Daniel Jackson. 2016. Object spreadsheets: a new computational model for end-user development of data-centric web applications. In *Proceedings of the 2016 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software* (Amsterdam, Netherlands) *(Onward! 2016)*. Association for Computing Machinery, New York, NY, USA, 112–127. doi:10.1145/2986012.2986018

[30] Cyrus Omar, Ian Voysey, Ravi Chugh, and Matthew A. Hammer. 2019. Live functional programming with typed holes. *Proc. ACM Program. Lang.* 3, POPL, Article 14 (Jan. 2019), 32 pages. doi:10.1145/3290327

[31] TOMAS PETRICEK. 2021. Composable data visualizations. *Journal of Functional Programming* 31 (2021), e13. doi:10.1017/S0956796821000046

[32] Tomas Petricek, Dominic Orchard, and Alan Mycroft. 2013. Coeffects: Unified Static Analysis of Context-Dependence. In *Automata, Languages, and Programming*, Fedor V. Fomin, Rūsiņš Freivalds, Marta Kwiatkowska, and David Peleg (Eds.). Springer Berlin Heidelberg, 385–397.

[33] Tomas Petricek, Dominic Orchard, and Alan Mycroft. 2014. Coeffects: a calculus of context-dependent computation. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming* (Gothenburg, Sweden) *(ICFP '14)*. Association for Computing Machinery, New York, NY, USA, 123–135. doi:10.1145/2628136.2628160

[34] Peter Sestoft and Jens Zeilund Sørensen. 2013. Sheet-Defined Functions: Implementation and Initial Evaluation. In *End-User Development*, Yvonne Dittrich, Margaret Burnett, Anders Mørch, and David Redmiles (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 88–103.

[35] The Center for Connected Learning and Computer-Based Modeling. 2025. *NetLogo Models Library*. https://ccl.northwestern.edu/netlogo/models/ Accessed: 25 February 2026.

[36] Zhanyong Wan and Paul Hudak. 2000. Functional reactive programming from first principles. In *Proceedings of the ACM SIGPLAN 2000 Conference on Programming Language Design and Implementation* (Vancouver, British Columbia, Canada) *(PLDI '00)*. Association for Computing Machinery, New York, NY, USA, 242–252. doi:10.1145/349299.349331

[37] Leland Wilkinson. 2012. *The Grammar of Graphics*. Springer Berlin Heidelberg, 375–414. doi:10.1007/978-3-642-21551-3_13

[38] Jack Williams, Nima Joharizadeh, Andrew D. Gordon, and Advait Sarkar. 2020. Higher-Order Spreadsheets with Spilled Arrays. In *Programming Languages and Systems - 29th European Symposium on Programming, ESOP 2020 (Lecture Notes in Computer Science, Vol. 12075)*, Peter Müller (Ed.). Springer, 743–769. doi:10.1007/978-3-030-44914-8_27

[39] A.G. Yoder and D.L. Cohn. 1994. Real spreadsheets for real programmers. In *Proceedings of 1994 IEEE International Conference on Computer Languages (ICCL'94)*. 20–30. doi:10.1109/ICCL.1994.288396